04 Assignment4 Tracking Rest

ShipTrack (shipment1212). TeamID: TODO | Name/Roll: TODO | Date: 2026-09-08.

Assignment 4 — Tracking Service in REST (NOT Payments)

TeamID: TODO | Name/Roll: TODO

Worked example: services/tracking-service/ (Delivery/Orders-history analogue). Stack is JS/Express + Jest (spec's models.py/store.py, pytest are Python names — JS equivalents below).

Part A — Model

- A1 service: tracking-service, owns Mongo tracking_db only (src/models/trackingEventModel.js, src/db/connection.js). Boundary: docs/architecture.md:59.
- A2 SOAP-style ops (starting point): recordTrackingEvent(...), getTrackingHistory(...), getLatestStatus(...), listEventsPaginated(...) (src/services/trackingEventService.js:16-35).
- A3 nouns: tracking-events. No verbs in URLs: POST /api/v1/tracking/{trackingId}/events, GET .../events?cursor&limit, GET .../latest (openapi.yaml:13-56, src/routes/trackingRoutes.js:11-16).
- A4 resource table (from openapi.yaml:13-56):

Method	URL	What	Success	Failures
POST	/{trackingId}/events	record event	201	400
GET	/{trackingId}/events?cursor &limit	filtered list	200	(add 400/404 for strict pass)
GET	/{trackingId}/latest	latest event (sub-resource)	200	404
PATCH	/shipments/{id}/status (shipment link, 4th row)	state change	200	412/422

- A5 hard choice: recordEvent is a verb forced into POST /{trackingId}/events collection-create; rejected POST /recordTrackingEvent RPC and PUT /status overwrite (loses history). Append-only log preserves audit + makes list/latest trivial.

Part B — Contract

- B1: services/tracking-service/openapi.yaml:1-84 has info, servers [localhost:8005, gateway], paths, components.schemas; every op has requestBody/parameters + ≥2 responses. Live at http://localhost:8005/api-docs (src/app.js:11).
- B2: schemas once: RecordEventRequest, TrackingEvent, Error (:60-84), all via $ref (:24,27,44,55).
- B3: 400 BadRequest + 404 NotFound declared; GET .../events lists only 200 — add 400/404 for strict pass. SOAP unknown-country fault → 404, validation fault → 400.
- B4 validate: npx @redocly/cli lint services/tracking-service/openapi.yaml or upload to editor.swagger.io (expect 0 errors). Save output/screenshot.

Part C — Implement

- C1 layout: src/models/trackingEventModel.js (Mongoose), src/services/trackingEventService.js (=store), src/app.js, src/routes/trackingRoutes.js, src/controllers/trackingController.js, tests/tracking.routes.test.js. Real Mongo (stronger than in-process dict); no cross-service store import — only HTTP.
- C2 record vs representation: toResponse() (src/services/trackingEventService.js:4-14) stores _id, trackingId, status, location, note, recordedBy, timestamp → publishes string id, hides _id/v; recordedBy falls back to 'system' (trackingController.js:5); no credential leak.
- C3 four endpoints: create POST /:trackingId/events, single GET /:trackingId/latest, filtered list GET /:trackingId/events?cursor&limit, state-change = POST appends event (or cite shipment PATCH /{id}/status).
- C4 validate: validate(recordEventSchema) before controller (src/routes/trackingRoutes.js:11); schema src/middleware/schemas.js:1-13 (required status, enum, additionalProperties:false). Without it {"status":"NOT_A_STATUS"} would reach Mongoose (test :28-33 proves 400).
- C5 codes: 201 create (trackingController.js:13), 200 reads, 400 enum, 404 no-events (trackingEventService.js:33). Location/202/409/422 live in shipment-service (409 in common/src/idempotency.js:29, 412/422 in shipment openapi.yaml) — quote for full coverage.
- C6 one error shape: {error:{code,message,details}} via common/src/errors.js:1-53 + central errorHandler (src/app.js:16).
- C7 idempotency: common/src/idempotency.js:11-49 (key→response cache, same-key-different-body→409, Idempotent-Replay:true). Wired on shipment POST /shipments (openapi.yaml:29-30, frontend/src/api/shipments.js:6). Tracking POST is append-only — add 1-line idempotency() to trackingRoutes.js if examiner demands the header on THIS service.
- C8 tests (6 in Jest, not pytest): tests/tracking.routes.test.js:27-83 — 400 bad enum, 201 unauthenticated as system, 201 carrier recordedBy, 200 public list, 404 unknown latest, 200 latest. Run: npm test --workspace services/tracking-service (DB mocked).

Part D — Survive the network

- D1 call: shipment → tracking via shipment-service/src/clients/trackingClient.js:1-20 (baseURL: process.env.TRACKING_SERVICE_URL); tracking accepts internal no-JWT call (trackingRoutes.js:8-11 optionalAuthenticate). Also shipment→pricing, pricing→location,

graphql→shipment/tracking/user.
- D2 harden: common/src/httpClient.js:1-71 — timeout 3000ms, axios-retry exponential backoff, retry only network/idempotent/5xx (never 4xx), Opossum breaker (50%, 10s reset), X-Correlation-Id.
- D3 fallback: trackingClient.js:9 fallback: () => null = best-effort: status change succeeds even if tracking down; degrading to delayed history is right, failing the shipment write would punish the customer for an analytics-side outage.

Curl transcript (run with -i)

```
curl -i -X POST http://localhost:8005/api/v1/tracking/STKABC123/events -H 'content-type: application/json' -d '{"status":"CR
curl -i http://localhost:8005/api/v1/tracking/STKABC123/events   # 200 list
curl -i http://localhost:8005/api/v1/tracking/UNKNOWN/latest     # 404
curl -i -X POST http://localhost:8005/api/v1/tracking/STKABC123/events -H 'content-type: application/json' -d '{"status":"NO
```

5 NOTES answers

1. wc -l location/.../countryWsdl.js tracking/openapi.yaml (≈61 vs 84). WSDL-only: (a) SOAP binding/transport + soapAction, (b) service/port address. OpenAPI instead has servers/paths.
2. soap:Fault (Client, UnknownCountry) → 404 {"error":{"code":"NOT_FOUND","message":"No tracking events found for UNKNOWN"}}. 200-with-error poisons caches and disables retries.
3. UDDI publish→git+image+/api-docs/?wsdl; find→env+gateway+Swagger; bind→build-time openapi.yaml/createClientAsync(wsdlUrl). Taken over by env, gateway routing, OpenAPI/Swagger, WSDL fetch.
4. Enforcer = validate(recordEventSchema) (trackingRoutes.js:11, AJV). Without it bad enum/oversized note reaches Mongo.
5. SOAP still wins for money/reference data across trust boundaries — buying WS-Security signing/encryption + strict versioning; REST+JWT+idempotency only approximates it.